

GradientBoostingClassifier#

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>

Description:

Gradient Boosting for classification. This algorithm builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes_` regression trees are fit on the negative gradient of the loss function, e.g. binary or multiclass log loss. Binary classification is a special case where only a single regression tree is induced. `HistGradientBoostingClassifier` is a much faster variant of this algorithm for intermediate and large datasets (`n_samples >= 10_000`) and supports monotonic constraints. Read more in the User Guide.

Function Signature

```
class sklearn.ensemble.GradientBoostingClassifier(*,  
loss='log_loss', learning_rate=0.1, n_estimators=100,  
subsample=1.0, criterion='friedman_mse',  
min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_depth=3,  
min_impurity_decrease=0.0, init=None, random_state=None,
```

```
max_features=None, verbose=0, max_leaf_nodes=None,  
warm_start=False, validation_fraction=0.1,  
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)[source]#
```

Parameters:

loss{'log_loss', 'exponential'}, default='log_loss'

learning_ratefloat, default=0.1

n_estimatorsint, default=100

```
class sklearn.ensemble.GradientBoostingClassifier(*,  
loss='log_loss', learning_rate=0.1, n_estimators=100,  
subsample=1.0, criterion='friedman_mse',  
min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_depth=3,  
min_impurity_decrease=0.0, init=None, random_state=None,  
max_features=None, verbose=0, max_leaf_nodes=None,  
warm_start=False, validation_fraction=0.1,  
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)[source]#
```

Parameters:

loss{'log_loss', 'exponential'}, default='log_loss'

learning_ratefloat, default=0.1

n_estimatorsint, default=100

apply(X)[source]#

Parameters:

X{array-like, sparse matrix} of shape (n_samples,
n_features)

Returns:

X_leavesarray-like of shape (n_samples, n_estimators,
n_classes)

Parameters

`tolfloat, default=1e`

Tolerance for the early stopping. When the loss is not improving by at least `tol` for `n_iter_no_change` iterations (if set to a number), the training stops. Values must be in the range `[0.0, inf)`. Added in version 0.20.

`ccp_alphanon`

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. Values must be in the range `[0.0, inf)`. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning. Added in version 0.22.

`X{array`

The input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted to a sparse `csr_matrix`.

`X_leavesarray`

For each datapoint x in X and for each tree in the ensemble, return the index of the leaf x ends up in each estimator. In the case of binary classification `n_classes` is 1.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`yarray`

Target values (strings or integers in classification, real numbers in regression)
For classification, labels must correspond to classes.

`sample_weightarray`

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. In the case of classification, splits are also ignored if they would result in any single class carrying a negative weight in either child node.

`paramsdict`

Parameter names mapped to their values.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`Xarray`

Test samples.

`yarray`

True labels for X.

`sample_weightarray`

Sample weights.

`**paramsdict`

Estimator parameters.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

Code Examples

```
N_t / N * (impurity - N_t_R / N_t * right_impurity
           - N_t_L / N_t * left_impurity)
```

```
>>> from sklearn.datasets import make_hastie_10_2
>>> from sklearn.ensemble import GradientBoostingClassifier
```

```
>>> X, y = make_hastie_10_2(random_state=0)
>>> X_train, X_test = X[:2000], X[2000:]
>>> y_train, y_test = y[:2000], y[2000:]
```

```
>>> clf = GradientBoostingClassifier(n_estimators=100,
learning_rate=1.0,
...     max_depth=1, random_state=0).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.913
```

Notes & Warnings

NOTE

See also `HistGradientBoostingClassifier` Histogram-based Gradient Boosting Classification Tree. `sklearn.tree.DecisionTreeClassifier` A decision tree classifier. `RandomForestClassifier` A meta-estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. `AdaBoostClassifier` A meta-estimator that begins by fitting a classifier on the original dataset and then fits additional copies of the classifier on the same dataset where the weights of incorrectly classified instances are adjusted such that subsequent classifiers focus more on difficult cases.

Detailed Documentation

Documentation

Gradient Boosting for classification.

This algorithm builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes_` regression trees are fit on the negative gradient of the loss function, e.g. binary or multiclass log loss. Binary classification is a special case where only a single regression tree is induced.

`HistGradientBoostingClassifier` is a much faster variant of this algorithm for intermediate and large datasets (`n_samples` $\geq 10\,000$) and supports monotonic constraints.

Read more in the User Guide.

The loss function to be optimized. 'log_loss' refers to binomial and multinomial deviance, the same as used in logistic regression. It is a good choice for classification with probabilistic outputs. For loss 'exponential', gradient boosting recovers the AdaBoost algorithm.

Learning rate shrinks the contribution of each tree by learning_rate. There is a trade-off between learning_rate and n_estimators. Values must be in the range [0.0, inf).

For an example of the effects of this parameter and its interaction with subsample, see Gradient Boosting regularization.

The number of boosting stages to perform. Gradient boosting is fairly robust to over-fitting so a large number usually results in better performance. Values must be in the range [1, inf).

The fraction of samples to be used for fitting the individual base learners. If smaller than 1.0 this results in Stochastic Gradient Boosting. subsample interacts with the parameter n_estimators. Choosing subsample < 1.0 leads to a reduction of variance and an increase in bias. Values must be in the range (0.0, 1.0].

The function to measure the quality of a split. Supported criteria are 'friedman_mse' for the mean squared error with improvement score by Friedman, 'squared_error' for mean squared error. The default value of 'friedman_mse' is generally the best as it can provide a better approximation in some cases.

Added in version 0.18.

The minimum number of samples required to split an internal node:

If int, values must be in the range [2, inf).

If float, values must be in the range (0.0, 1.0] and `min_samples_split` will be `ceil(min_samples_split * n_samples)`.

Changed in version 0.18: Added float values for fractions.

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

If int, values must be in the range [1, inf).

If float, values must be in the range (0.0, 1.0) and `min_samples_leaf` will be `ceil(min_samples_leaf * n_samples)`.

Changed in version 0.18: Added float values for fractions.

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided. Values must be in the range [0.0, 0.5].

Maximum depth of the individual regression estimators. The maximum depth limits the number of nodes in the tree. Tune this parameter for best performance; the best value depends on the interaction of the input variables. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples. If int, values must be in the range [1, inf).

A node will be split if this split induces a decrease of the impurity greater than or equal to this value. Values must be in the range [0.0, inf).

The weighted impurity decrease equation is the following:

where N is the total number of samples, N_t is the number of samples at the current node, N_{t_L} is the number of samples in the left child, and N_{t_R} is the number of samples in the right child.

N , N_t , N_{t_R} and N_{t_L} all refer to the weighted sum, if `sample_weight` is passed.

Added in version 0.19.

An estimator object that is used to compute the initial predictions. `init` has to provide `fit` and `predict_proba`. If 'zero', the initial raw predictions are set to zero. By default, a `DummyEstimator` predicting the classes priors is used.

Controls the random seed given to each Tree estimator at each boosting iteration. In addition, it controls the random permutation of the features at each split (see Notes for more details). It also controls the random splitting of the training data to obtain a validation set if `n_iter_no_change` is not `None`. Pass an int for reproducible output across multiple function calls. See Glossary.

The number of features to consider when looking for the best split:

If int, values must be in the range $[1, \infty)$.

If float, values must be in the range $(0.0, 1.0]$ and the features considered at each split will be $\max(1, \text{int}(\text{max_features} * \text{n_features_in_}))$.

If 'sqrt', then $\text{max_features} = \sqrt{\text{n_features}}$.

If 'log2', then $\text{max_features} = \log_2(\text{n_features})$.

If `None`, then $\text{max_features} = \text{n_features}$.

Choosing $\text{max_features} < \text{n_features}$ leads to a reduction of variance and an increase in bias.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

Enable verbose output. If 1 then it prints progress and performance once in a while (the more trees the lower the frequency). If greater than 1 then it prints progress and performance for every tree. Values must be in the range $[0, \infty)$.

Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. Values must be in the range $[2, \infty)$. If None, then unlimited number of leaf nodes.

When set to True, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just erase the previous solution. See the Glossary.

The proportion of training data to set aside as validation set for early stopping. Values must be in the range $(0.0, 1.0)$. Only used if `n_iter_no_change` is set to an integer.

Added in version 0.20.

`n_iter_no_change` is used to decide if early stopping will be used to terminate training when validation score is not improving. By default it is set to None to disable early stopping. If set to a number, it will set aside `validation_fraction` size of the training data as validation and terminate training when validation score is not improving in all of the previous `n_iter_no_change` numbers of iterations. The split is stratified. Values must be in the range $[1, \infty)$. See Early stopping in Gradient Boosting.

Added in version 0.20.

Tolerance for the early stopping. When the loss is not improving by at least `tol` for `n_iter_no_change` iterations (if set to a number), the training stops. Values must be in the range $[0.0, \infty)$.

Added in version 0.20.

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. Values must be in the range `[0.0, inf)`. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning.

Added in version 0.22.

The number of estimators as selected by early stopping (if `n_iter_no_change` is specified). Otherwise it is set to `n_estimators`.

Added in version 0.20.

The number of trees that are built at each iteration. For binary classifiers, this is always 1.

Added in version 1.4.0.

The impurity-based feature importances.

The improvement in loss on the out-of-bag samples relative to the previous iteration. `oob_improvement_[0]` is the improvement in loss of the first stage over the init estimator. Only available if `subsample < 1.0`.

The full history of the loss values on the out-of-bag samples. Only available if `subsample < 1.0`.

Added in version 1.3.

The last value of the loss on the out-of-bag samples. It is the same as `oob_scores_-1]`. Only available if `subsample < 1.0`.

Added in version 1.3.

The i-th score `train_score_[i]` is the loss of the model at iteration i on the in-bag sample. If `subsample == 1` this is the loss on the training data.

The estimator that provides the initial predictions. Set via the `init` argument.

The collection of fitted sub-estimators. `n_trees_per_iteration_` is 1 for binary classification, otherwise `n_classes`.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

The number of classes.

The inferred value of `max_features`.

Histogram-based Gradient Boosting Classification Tree.

A decision tree classifier.

A meta-estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control overfitting.

A meta-estimator that begins by fitting a classifier on the original dataset and then fits additional copies of the classifier on the same dataset where the weights of incorrectly

classified instances are adjusted such that subsequent classifiers focus more on difficult cases.

The features are always randomly permuted at each split. Therefore, the best found split may vary, even with the same training data and `max_features=n_features`, if the improvement of the criterion is identical for several splits enumerated during the search of the best split. To obtain a deterministic behaviour during fitting, `random_state` has to be fixed.

J. Friedman, Greedy Function Approximation: A Gradient Boosting Machine, The Annals of Statistics, Vol. 29, No. 5, 2001.

Friedman, Stochastic Gradient Boosting, 1999

T. Hastie, R. Tibshirani and J. Friedman. Elements of Statistical Learning Ed. 2, Springer, 2009.

The following example shows how to fit a gradient boosting classifier with 100 decision stumps as weak learners.

Apply trees in the ensemble to X, return leaf indices.

Added in version 0.17.

The input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted to a sparse `csr_matrix`.

For each datapoint `x` in `X` and for each tree in the ensemble, return the index of the leaf `x` ends up in each estimator. In the case of binary classification `n_classes` is 1.

Compute the decision function of `X`.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse

matrix is provided to a sparse csr_matrix.

The decision function of the input samples, which corresponds to the raw values predicted from the trees of the ensemble . The order of the classes corresponds to that in the attribute classes_. Regression and binary classification produce an array of shape (n_samples,).

Fit the gradient boosting model.

The input samples. Internally, it will be converted to dtype=np.float32 and if a sparse matrix is provided to a sparse csr_matrix.

Target values (strings or integers in classification, real numbers in regression) For classification, labels must correspond to classes.

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. In the case of classification, splits are also ignored if they would result in any single class carrying a negative weight in either child node.

The monitor is called after each iteration with the current iteration, a reference to the estimator and the local variables of _fit_stages as keyword arguments callable(i, self, locals()). If the callable returns True the fitting procedure is stopped. The monitor can be used for various things such as computing held-out estimates, early stopping, model introspect, and snapshotting.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for t